# Inventora Sistem Enterprise Resource Planning dan Pengadaan

## TUGAS KELOMPOK

## Evaluasi Class Diagram dan Kerangka Kode serta Pemeriksaan Login



**Anggota Kelompok**

1. Ichya Ulumiddiin (103112400076)
2. M. Zaky Mubarok (103112400073)
3. Lutfi Shidqi Mardian (103112400077)
4. Daffa Tsaqifna Fauztsany (103112400032)
5. Rizky Iffat Venardi (2311102301)

**FAKULTAS INFORMATIKA**

**TELKOM UNIVERSITY PURWOKERTO**

**2026**

**Daftar isi**

1. Pendahuluan....3

2. Evaluasi model awal....3

3. Rancangan Class Diagram....5

4. Atribut dan operasi kelas....11

5. Kerangka kode TypeScript....13

6. Alur dan status login....14

7. Hasil pemeriksaan....16

8. Pemenuhan tugas dan kesimpulan....17

Lampiran A. Deklarasi tipe data....18

Lampiran B. Kerangka kelas....26

Lampiran C. Berkas dan cara pemeriksaan....33

## 1. Pendahuluan

Model awal Inventora sudah menggambarkan hubungan data pengadaan, tetapi belum memenuhi isi Class Diagram UML karena belum memuat tipe atribut, visibilitas, dan operasi. Laporan ini mengevaluasi model tersebut dan menyusun rancangan kelas berdasarkan kode Inventora. Hasilnya berupa 11 kerangka kelas, 15 interface data, dan 20 relasi yang dicantumkan secara eksplisit.

### 1.1 Tujuan dan lingkup

Evaluasi mencakup pengadaan, penerimaan barang, pembayaran, sesi pengguna, serta layanan inventori, SDM dan payroll, keuangan, dan langganan. Setiap atribut mempunyai tipe data; operasi mempunyai parameter, tipe hasil, dan access modifier. Kerangka kode dibuat tanpa implementasi isi method sebagaimana diminta pada tugas.

Inventora menggunakan React dengan komponen fungsi, interface TypeScript, Firebase Authentication, dan Cloud Firestore. Kelas layanan dalam laporan merupakan usulan pembagian tanggung jawab. Kelas tersebut belum dihubungkan ke komponen React. Login aplikasi tetap memakai implementasi yang sudah ada.

### 1.2 Dasar pemeriksaan

Sumber utama adalah src/types.ts, src/firebase.ts, komponen Login, App, ProcurementView, PurchaseUnitView, HRView, InventoryView, FinanceView, BillingSubscriptionView, modul authorization, procurement, dan purchaseOrderStore, serta firestore.rules. Dokumen DPPL yang diberikan menjadi acuan format dan pembanding model awal. Laporan basis data Firebase sebelumnya dipakai untuk mencocokkan bentuk dokumen serta bukti pengujian emulator.

Pemeriksaan dilakukan pada 21 Oktober 2026. Hasil pemeriksaan kode dibedakan dari pengujian interaksi layar dan login menggunakan akun Google asli.

## 2. Evaluasi model awal

### 2.1 Kelengkapan diagram awal

Gambar 2 pada dokumen DPPL merupakan model hubungan entitas atau ERD konseptual. Model itu menampilkan Product, User, Role, PurchaseOrder, POLineItem, GoodsReceiptNote, PaymentTransaction, serta beberapa target pengembangan. Hubungan tersebut membantu membaca domain, tetapi belum menyatakan struktur kelas beserta perilakunya.

| Aspek | Temuan dan keputusan |
|---|---|
| Jenis model | ERD belum dapat disebut Class Diagram lengkap. Rancangan baru membedakan interface data dari kelas berperilaku. |
| Atribut | Tipe, atribut opsional, dan visibilitas belum lengkap. Deklarasi pada Lampiran A dan B melengkapinya. |
| Method | Operasi dan tanda visibilitas belum tersedia. Kerangka kelas menuliskan public dan private secara eksplisit. |
| Relasi | PO dengan item, GRN, dan paymentInfo merupakan komposisi data tertanam. Riwayat PaymentTransaction dibedakan dari paymentInfo. |
| Fitur target | SaleTransaction, SaleLine, dan AuditEvent pada model awal belum menjadi implementasi POS atau audit cloud yang terkonfirmasi. |
| Pewarisan | Peran tidak dijadikan subclass pengguna. Peran adalah nilai Role yang diperiksa oleh kebijakan otorisasi. |

### 2.2 Konsistensi struktur dan tipe

PurchaseOrder.status masih bertipe POStatus | string. Union tersebut menerima string di luar daftar status resmi, sehingga pembatasan tipe belum kuat. color juga menerima string bebas, sedangkan lineItems masih any[]. Rancangan mempertahankan bentuk aktual agar laporan tidak menggambarkan perubahan yang belum dilakukan. Perbaikan berikutnya adalah membatasi status dan mengganti lineItems dengan tipe yang dinyatakan.

items dan lineItems menyimpan representasi rincian barang yang bertumpang tindih. Rumus subtotal dan totalPrice perlu dipusatkan pada satu bentuk data, kemudian format lama dimigrasikan. Jumlah uang memakai number dengan konvensi rupiah bilangan bulat; tanggal pada tipe saat ini memakai string. Validasi nilai dan format tetap diperlukan ketika data masuk.

Atribut items bersifat opsional untuk kompatibilitas data lama. Karena itu kardinalitas pada model aktual ditulis 0..*. Formulir pembuatan PO baru mewajibkan sedikitnya satu item. Kardinalitas rancangan bisnis dapat diperketat menjadi 1..* setelah data lama dimigrasikan.

### 2.3 Kenormalan dan batas penyimpanan

Penilaian kenormalan Class Diagram mencakup tanggung jawab, enkapsulasi, jenis relasi, kardinalitas, dan konsistensi tipe. Bentuk normal relasional tidak diterapkan langsung pada

dokumen Firestore. Penanaman items, grn, dan paymentInfo di dalam satu PO mengikuti kebutuhan pembacaan dan transaksi pengadaan.

Koleksi Firestore yang aktif pada alur yang diperiksa adalah purchaseOrders. Inventori, SDM, payroll, invoice keuangan, dan langganan menggunakan penyimpanan browser atau data konfigurasi. Interface tidak otomatis menjadi koleksi. activities pada blueprint belum menjadi audit cloud yang aktif dan aturan saat ini menolak akses yang tidak dinyatakan.

Relasi Employee dengan PayrollDisbursementRecord adalah asosiasi riwayat, sehingga penghapusan pegawai tidak seharusnya menghapus bukti gaji. EmployeeSalary menjadi bagian Employee. Tidak ada foreign key bawaan Firestore; referensi poId dan identitas item harus divalidasi oleh aplikasi serta Security Rules sesuai operasi yang diizinkan.

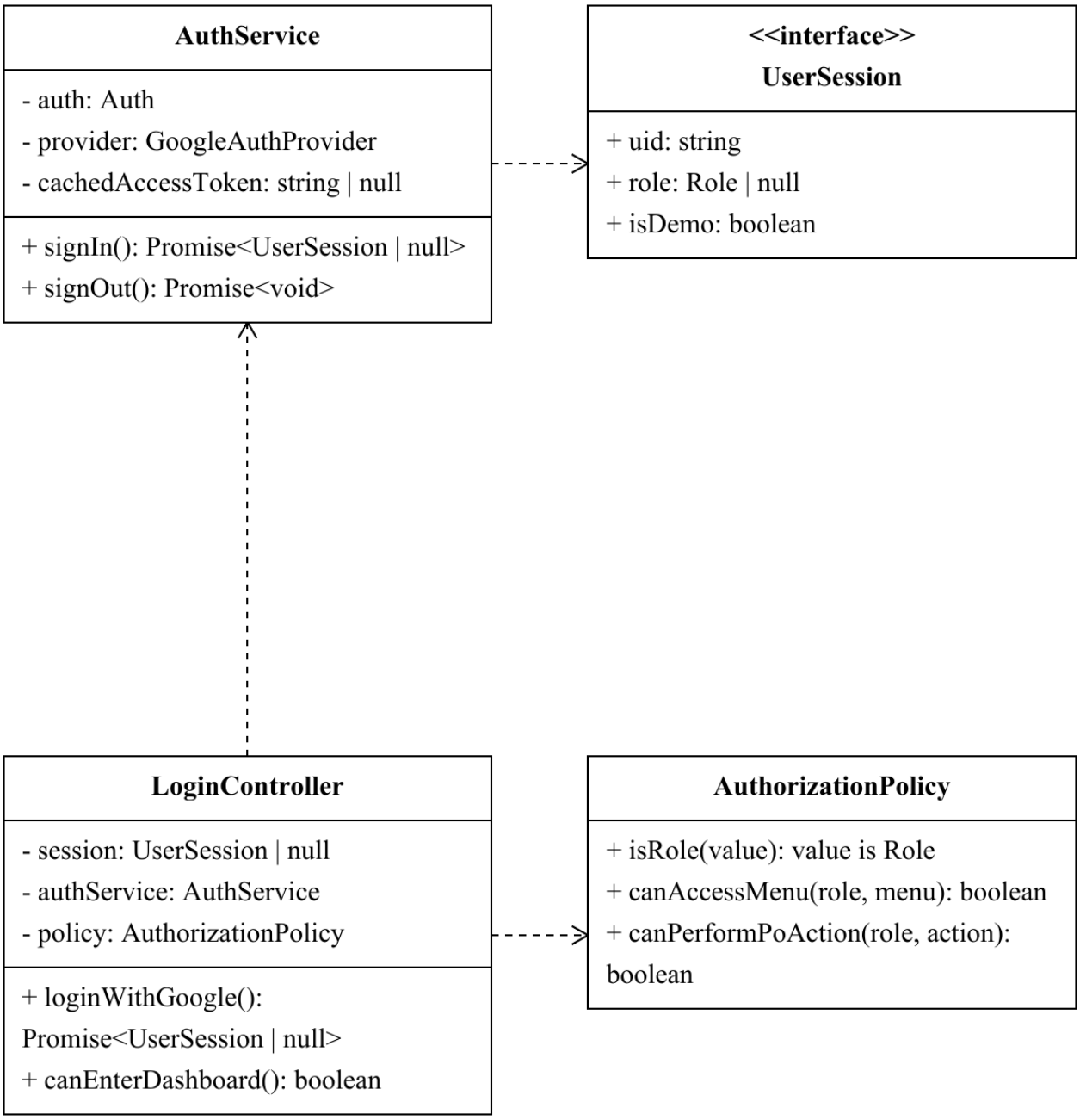
## 3. Rancangan Class Diagram

### 3.1 Notasi dan pembacaan

Tanda + berarti public, - berarti private, dan # berarti protected. Interface TypeScript mempunyai anggota yang dapat dibaca secara publik; kata public tidak dituliskan di dalam interface karena sintaks TypeScript tidak mengizinkannya. Tanda ? menunjukkan atribut atau parameter opsional. Tidak ada protected maupun pewarisan yang dipaksakan pada rancangan ini.

Empat gambar berikut merupakan tampilan ringkas dari model yang sama. Beberapa anggota ditampilkan untuk memperlihatkan tanggung jawab; seluruh atribut dan tanda tangan method tersedia pada Lampiran A dan B. Sumber PlantUML memuat semua anggota dan 20 relasi.

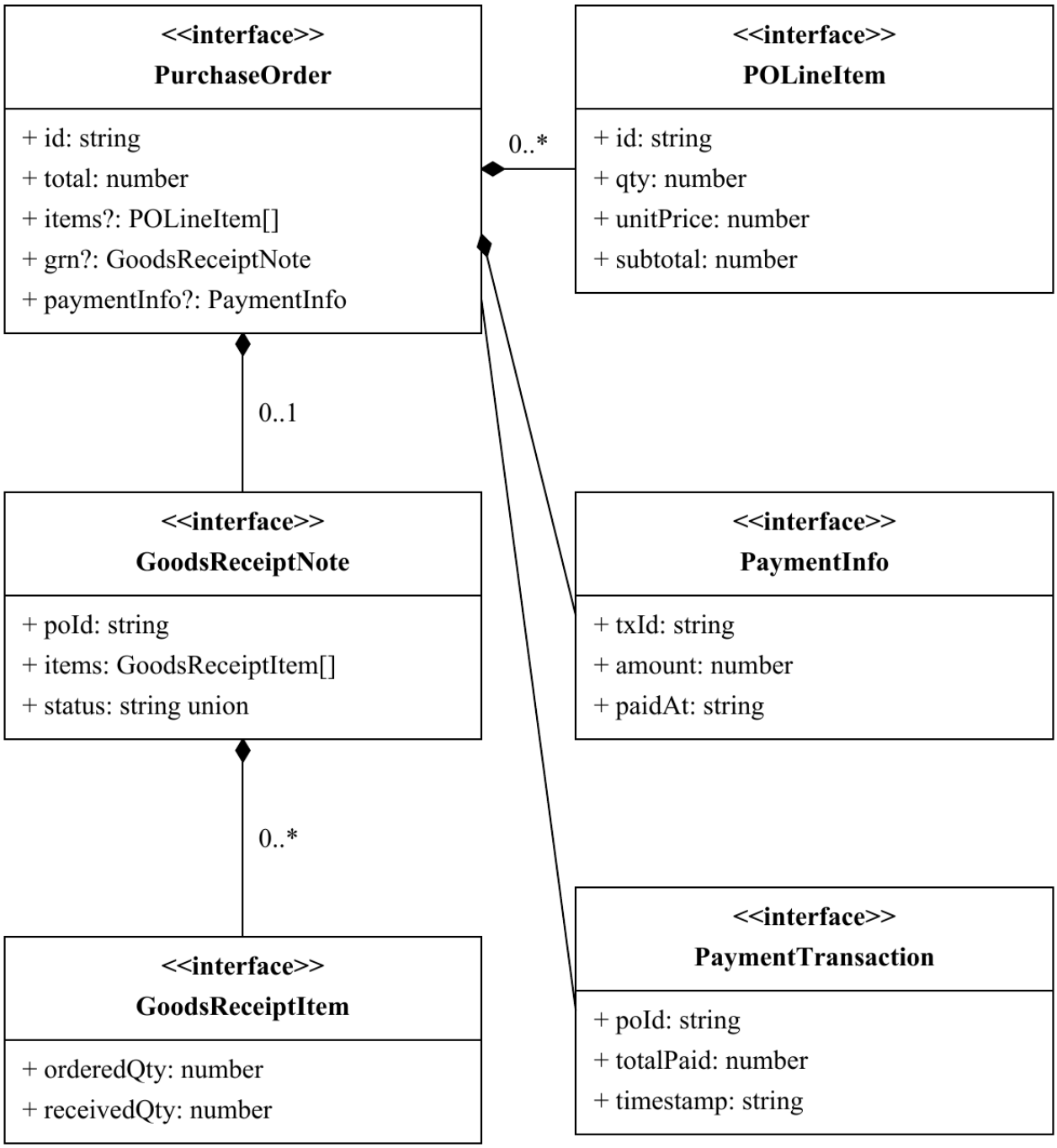
### 3.2 Sesi dan akses pengguna




Gambar 1. Kelas sesi dan kebijakan akses

Sesi berasal dari Firebase Authentication. Role adalah nilai custom claim; interface UserSession merapikan data yang dipakai aplikasi.

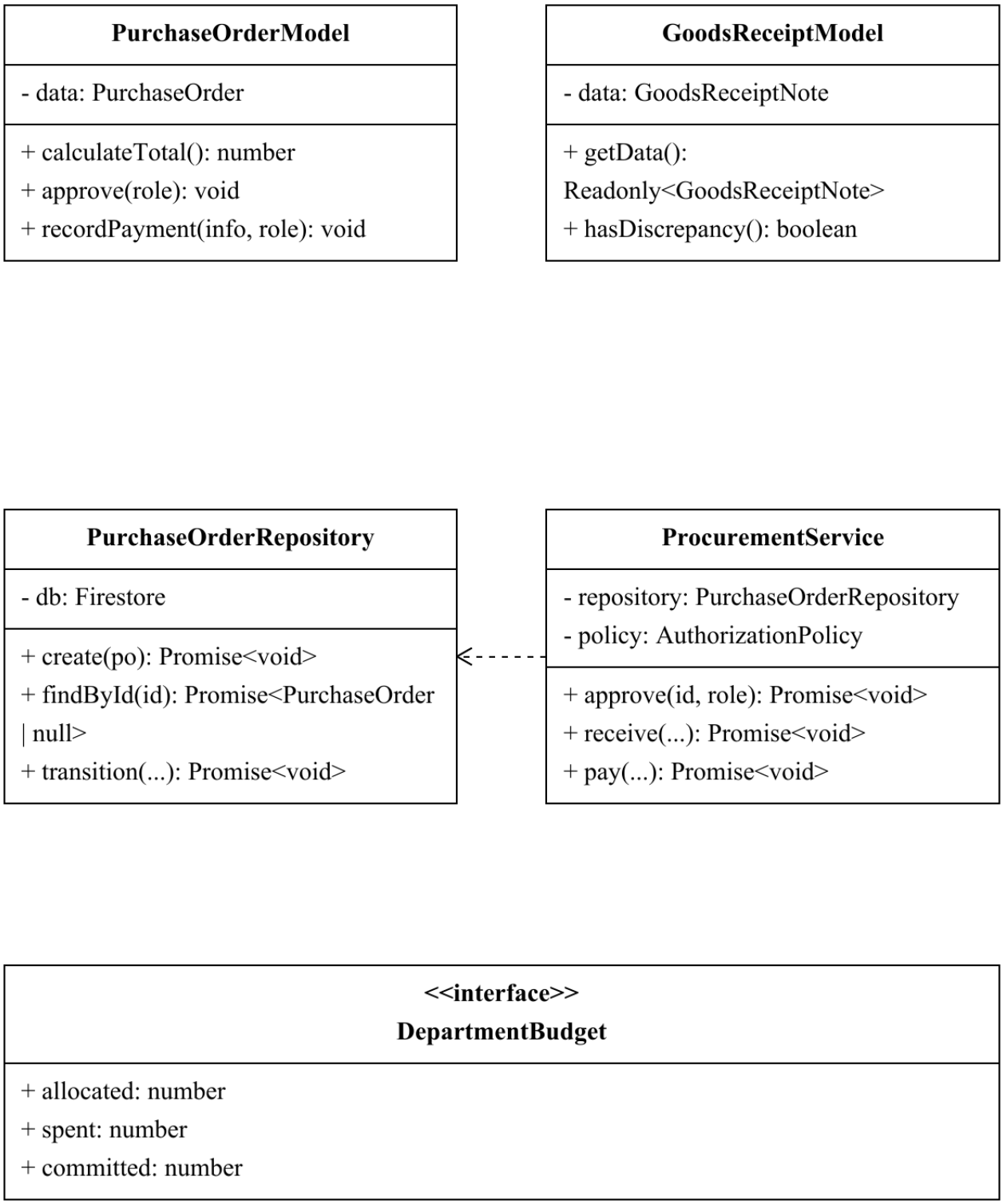
### 3.3 Data pengadaan dan penerimaan




Gambar 2. Struktur data pengadaan

Belah ketupat hitam berada pada objek pemilik. Angka pada ujung bagian menunjukkan jumlah bagian per satu pemilik. Relasi paymentInfo dan riwayat pembayaran dilengkapi pada tabel relasi.

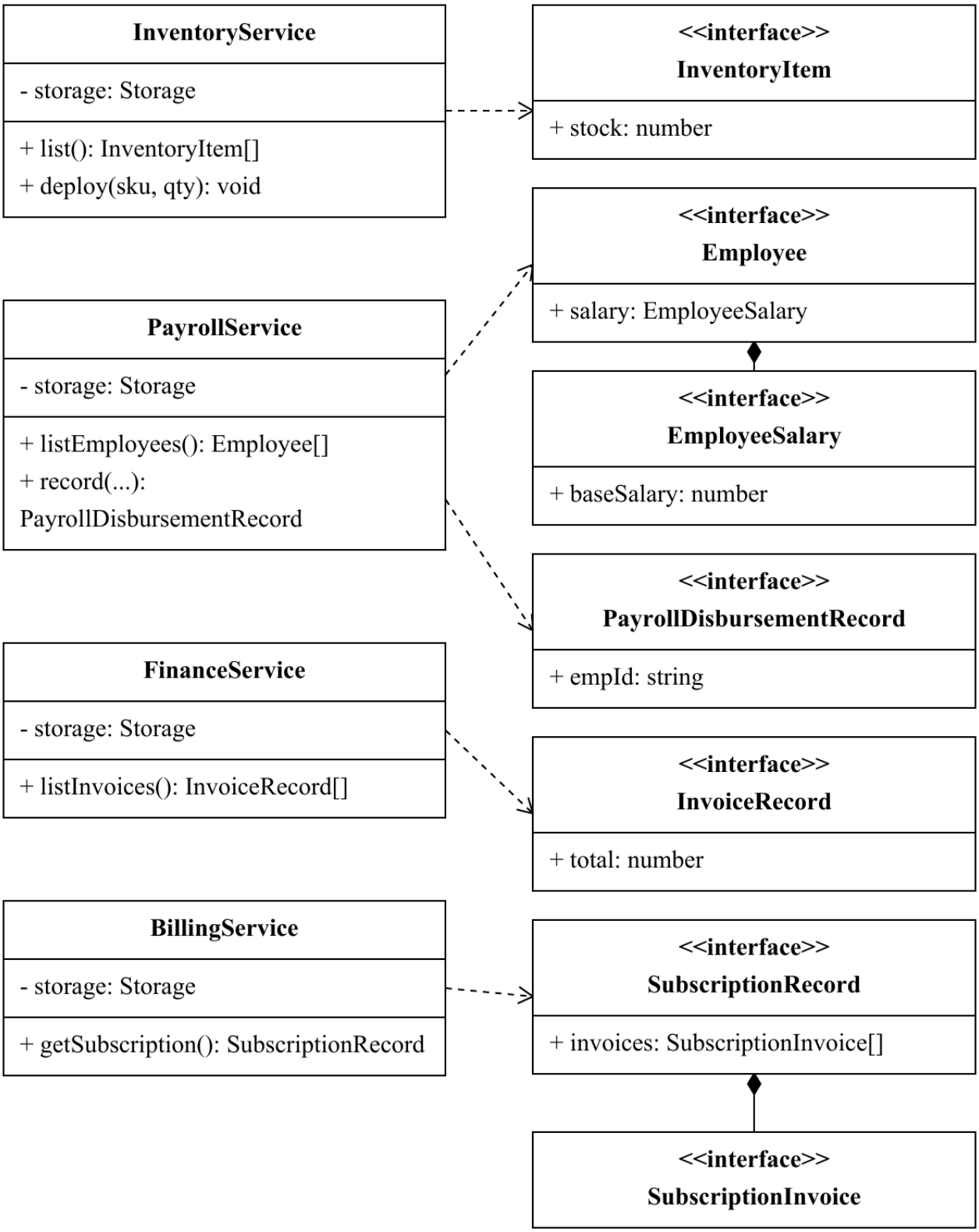
### 3.4 Model dan layanan pengadaan




Gambar 3. Pembagian tanggung jawab pengadaan

Model menyatakan perilaku satu objek, layanan mengatur tindakan, dan repository mengakses Firestore. DepartmentBudget ditampilkan sebagai data pendukung; relasi identitasnya belum tersedia pada kode.

### 3.5 Layanan dan data modul lokal




Gambar 4. Layanan dengan penyimpanan browser

Layanan lokal diusulkan untuk memisahkan logika dari komponen. Kotak SubscriptionInvoice adalah interface. Ketergantungan SDM, keuangan, dan langganan dibaca bersama tabel relasi agar garis tidak bertumpuk.

### 3.6 Relasi dan kardinalitas

| Relasi | Jenis | Kardinalitas |
|---|---|---|
| PurchaseOrder ke POLineItem | Komposisi | 1 ke 0..* |
| PurchaseOrder ke GoodsReceiptNote | Komposisi | 1 ke 0..1 |
| PurchaseOrder ke PaymentInfo | Komposisi | 1 ke 0..1 |
| GoodsReceiptNote ke GoodsReceiptItem | Komposisi | 1 ke 0..* |
| PurchaseOrder ke PaymentTransaction | Asosiasi logis riwayat lokal | 1 ke 0..* |
| Employee ke EmployeeSalary | Komposisi | 1 ke 1 |
| Employee ke PayrollDisbursementRecord | Asosiasi historis | 1 ke 0..* |
| SubscriptionRecord ke SubscriptionInvoice | Komposisi | 1 ke 0..* |
| PurchaseOrderModel ke PurchaseOrder | Asosiasi objek data | 1 ke 1 |
| GoodsReceiptModel ke GoodsReceiptNote | Asosiasi objek data | 1 ke 1 |

PaymentTransaction memuat poId sebagai penghubung riwayat pembayaran yang disimpan lokal. Dokumen PO sendiri mempunyai paling banyak satu map paymentInfo, sehingga relasi 1 ke 0..* pada riwayat tidak berarti aplikasi mendukung angsuran atau beberapa pembayaran pada satu PO. Aturan alur saat ini mencegah pembayaran ulang.

| Pemakai | Ketergantungan |
|---|---|
| LoginController | AuthService dan AuthorizationPolicy |
| AuthService | UserSession |
| ProcurementService | PurchaseOrderRepository dan AuthorizationPolicy |
| InventoryService | InventoryItem |
| PayrollService | Employee dan PayrollDisbursementRecord |
| FinanceService | InvoiceRecord |
| BillingService | SubscriptionRecord |

Ketergantungan menunjukkan pemakaian tipe atau layanan dan tidak menyatakan jumlah objek yang disimpan. Referensi SDK seperti Auth, Firestore, GoogleAuthProvider, Unsubscribe, dan Storage adalah tipe eksternal. DepartmentBudget merupakan tipe data pendukung perhitungan anggaran; model tidak mengarang relasi foreign key yang belum tersedia pada kode.

## 4. Atribut dan operasi kelas

### 4.1 Enkapsulasi dan tipe atribut

Data internal kelas dibuat private. Referensi auth, db, storage, repository, policy, dan data pada constructor memakai readonly agar referensinya tidak diganti setelah objek dibentuk. Readonly tersebut bersifat dangkal; nilai bertingkat masih memerlukan salinan atau pembatasan tambahan apabila kelas benar-benar diimplementasikan. Operasi yang dipakai pemanggil diberi public.

AuthService menyimpan provider dan token Google pada memori. LoginController menyimpan sesi. Repository memusatkan akses Firestore. ProcurementService memusatkan tindakan alur pengadaan; PurchaseOrderModel menangani aturan pada satu objek. Otorisasi antarlayanan dan pembatasan di Firestore tetap harus berlaku ketika kerangka diimplementasikan.

| Kelompok tipe | Isi dan ketentuan |
|---|---|
| Sesi | UserSession menyimpan uid, email, displayName, role, dan isDemo. Tidak menyimpan kata sandi Google. |
| Pengadaan | PurchaseOrder, POLineItem, GoodsReceiptNote, GoodsReceiptItem, PaymentInfo, PaymentTransaction, DepartmentBudget. |
| SDM | Employee, EmployeeSalary, PayrollDisbursementRecord. Riwayat payroll menyimpan komponen gaji pada periode pembayaran. |
| Modul lokal | InventoryItem, InvoiceRecord, SubscriptionInvoice, SubscriptionRecord. Nama tipe merapikan bentuk objek pada komponen. |

### 4.2 Kontrak operasi

| Kelas | Kontrak utama |
|---|---|
| AuthService | signIn menghasilkan sesi atau null jika dibatalkan; observe menghasilkan fungsi berhenti berlangganan; signOut mengakhiri sesi. |
| AuthorizationPolicy | Memeriksa Role, akses menu, dan tindakan PO. Hasil boolean atau type predicate. |
| LoginController | Mengatur login dan logout serta keputusan masuk dashboard dari sesi yang tersedia. |
| PurchaseOrderModel dan GoodsReceiptModel | Mengakses data, menghitung total, menyatakan transisi, dan memeriksa selisih penerimaan. |
| PurchaseOrderRepository | Membuat, membaca, mendengarkan perubahan, dan menyimpan transisi PO secara asinkron. |
| ProcurementService | Menjalankan create, approve, reject, receive, dan pay dengan identitas peran. |
| Layanan lokal | Inventori mengubah stok; payroll mencatat per pegawai dan periode; keuangan mencatat invoice; billing mengubah paket dan perpanjangan. |

Tanda tangan lengkap mencantumkan Promise untuk operasi asinkron, null atau undefined untuk hasil yang tidak tersedia, serta tipe input yang dinyatakan. Akses private pada mapUser membatasi pemetaan objek Firebase menjadi sesi internal. public constructor menunjukkan dependensi yang harus disediakan saat membentuk kelas.

### 4.3 Pemetaan ke implementasi sekarang

| Rancangan | Lokasi perilaku sekarang |
|---|---|
| AuthService | src/firebase.ts: googleSignIn, getAccessToken, logout; pengamatan token berada di App. |
| AuthorizationPolicy | src/lib/authorization.ts: isRole, canAccessMenu, canPerformPoAction. |

| Rancangan | Lokasi perilaku sekarang |
|---|---|
| LoginController | Komponen Login dan pengelolaan sesi pada App. |
| Model dan layanan pengadaan | src/lib/procurement.ts dan komponen pengadaan/pembelian. |
| PurchaseOrderRepository | src/lib/purchaseOrderStore.ts serta pembacaan/listener pada ProcurementView. |
| Layanan lokal | InventoryView, HRView, FinanceView, BillingSubscriptionView, dan helper penyimpanan yang dipakai komponen. |

**5. Kerangka kode TypeScript**

Kerangka disimpan pada docs/tubes-class-diagram/kerangka.ts. Setiap method berisi throw new Error dengan pesan Belum diimplementasikan. Penanda ini membuat kode dapat diperiksa oleh TypeScript tanpa mengarang perilaku method yang belum dibuat. Constructor hanya mendeklarasikan dependensi melalui parameter property.

Kerangka mengimpor tipe inti dari src/types.ts. Deklarasi Employee, EmployeeSalary, PayrollDisbursementRecord, dan SubscriptionTier disalin ke data-models.ts dari kode komponen agar pemeriksaan kerangka tidak perlu memuat implementasi antarmuka. Skrip susun-model.mjs membentuk ulang salinan tipe dan sumber diagram dari deklarasi proyek.

```
export class PurchaseOrderModel {
  public constructor(
    private readonly data: PurchaseOrder,
  ) {}
  public calculateTotal(): number {
    throw new Error('Belum diimplementasikan');
  }
}
```

Cuplikan di atas memperlihatkan pola deklarasi. Seluruh 11 kelas tercantum pada Lampiran B. Kerangka lulus pemeriksaan strict pada konfigurasi terpisah. Kelulusan kompilasi tidak berarti method telah diimplementasikan atau telah terintegrasi dengan React.

## 6. Alur dan status login

### 6.1 Alur antarmuka dan kode

Pengguna menekan tombol masuk dengan Google pada halaman Login. handleGoogleLogin memanggil googleSignIn, yang menjalankan signInWithPopup dari Firebase Authentication. Setelah berhasil, aplikasi menuju dashboard. Pembatalan popup menghasilkan null; kesalahan domain dan popup ditampilkan melalui penanganan kesalahan yang tersedia.

App mengamati perubahan melalui onIdTokenChanged, membaca getIdTokenResult, lalu memeriksa claims.role dengan isRole. Akun tanpa peran yang sah diarahkan ke tampilan akses belum ditetapkan. Empat peran yang dikenal adalah Super Admin, Warehouse Manager, Finance Manager, dan HR Manager. Peran berasal dari custom claim server, bukan pilihan bebas pengguna.

### 6.2 Hubungan autentikasi dan basis data

Firebase Authentication mengelola identitas pengguna. Firestore menerima permintaan dengan token autentikasi dan menerapkan firestore.rules pada purchaseOrders. Tindakan perubahan PO juga melewati persistPoTransition, yang memeriksa pengguna, peran, transisi, dan versi dokumen dalam transaksi. Penyembunyian menu di antarmuka tidak menggantikan aturan basis data.

ID token dipakai untuk identitas Firebase dan custom claim. OAuth access token Google dipakai untuk integrasi Google seperti Sheets dan Calendar; keduanya berbeda fungsi. Mode demo menggunakan inventora_demo_user di sessionStorage dan hanya mensimulasikan sesi lokal. Mode tersebut tidak membuktikan autentikasi Google atau izin Firestore cloud.

### 6.3 Status verifikasi login

| Lapisan | Status bukti |
|---|---|
| Antarmuka | Tombol dan handler login ada pada kode. Interaksi layar pada sesi ini belum diverifikasi. |
| Kode autentikasi | Popup Google, pembatalan, logout, pembacaan token, dan pemeriksaan claim ditemukan pada sumber. |
| Aturan akses | Pengujian regresi peran lulus. Bukti emulator terdahulu menunjukkan akses tanpa autentikasi dan peran HR ditolak. |
| Login Google asli | Belum diverifikasi melalui interaksi akun pada sesi ini. |
| Konfigurasi cloud | Provider Google, authorized domains, custom claim akun, dan rules yang terpasang perlu diperiksa pada lingkungan demonstrasi. |

Akses browser untuk memeriksa halaman lokal ditolak oleh kebijakan alat pada sesi penyusunan laporan. Oleh karena itu laporan tidak menyatakan login Google telah lulus pengujian dari antarmuka sampai basis data cloud. Pemeriksaan kode, regresi lokal, dan emulator dicatat sesuai batas bukti masing-masing.

### 6.4 Skenario demonstrasi yang masih diperlukan

| No | Skenario dan hasil yang diharapkan |
|---|---|
| 1 | Masuk dengan akun Google yang mempunyai custom claim sah; dashboard dan menu sesuai peran. |
| 2 | Muat ulang halaman; status autentikasi dipulihkan dan claim tetap diperiksa. |
| 3 | Masuk dengan akun tanpa claim; halaman akses belum ditetapkan dan tidak mendapat data PO. |
| 4 | Batalkan popup atau tolak akses; aplikasi tetap dapat digunakan dan pesan sesuai keadaan. |
| 5 | Uji operasi PO dengan peran yang berhak, lalu peran yang dilarang; Firestore menegakkan aturan. |
| 6 | Logout; sesi berakhir dan halaman yang memerlukan autentikasi tidak dapat diakses kembali tanpa login. |

## 7. Hasil pemeriksaan

### 7.1 Pemeriksaan pada penyusunan laporan

| Pemeriksaan | Hasil | Makna |
|---|---|---|
| TypeScript aplikasi | Lulus | tsc --noEmit pada konfigurasi proyek. |
| Regresi lokal | 15 lulus | Aturan bisnis dan helper yang disebutkan di bawah. |
| TypeScript kerangka | Lulus | strict dan noEmit pada tsconfig khusus kerangka. |
| Login Google asli | Belum diuji | Tidak ada bukti interaksi akun dan pembacaan cloud dari sesi ini. |

Lima belas regresi memeriksa tanggal bisnis Jakarta; pencegahan gaji ganda per pegawai dan periode; batas siklus langganan; langganan yang dijeda; ringkasan dashboard; hak akses peran; urutan PO dan pencegahan bayar ulang; nominal pembayaran; migrasi data pembayaran lama; antrean perubahan saat offline; pemulihan antrean rusak; konsistensi ekstraksi invoice; komponen riwayat slip gaji; backup dan reset; serta kegagalan penulisan penyimpanan lokal.

### 7.2 Bukti emulator dari tugas basis data sebelumnya

Berkas database/firestore/hasil-uji.json mencatat 18 pemeriksaan lulus pada 21 Oktober 2026 untuk project demo-inventora-tubes, endpoint 127.0.0.1:8085, dan lima dokumen purchaseOrders. Bukti tersebut berasal dari pengujian tugas basis data sebelumnya dan tidak dijalankan ulang dalam penyusunan laporan ini.

Skenario meliputi pembuatan PO, persetujuan, penolakan, penerimaan, pembayaran, serta penolakan akses dan transisi yang dilarang. Identitas uji memakai mock emulator. Hasil ini mendukung pemeriksaan Security Rules lokal dan tidak menjadi bukti keberhasilan login Google pada cloud.

### 7.3 Batas pemeriksaan

Pemeriksaan belum membuktikan seluruh interaksi layar, kondisi perangkat, aksesibilitas, konfigurasi cloud, atau semua bentuk data tidak valid. Nilai status bebas, any[], duplikasi rincian barang, dan penyimpanan lokal lintas perangkat masih menjadi pekerjaan lanjutan. Pemeriksaan strict hanya diterapkan pada kerangka terpisah; aplikasi diperiksa dengan konfigurasi TypeScript proyek yang tersedia.

## 8. Pemenuhan tugas dan kesimpulan

| Permintaan tugas | Pemenuhan |
|---|---|
| Evaluasi kelengkapan dan kenormalan | Bagian 2 menilai jenis model awal, tanggung jawab, konsistensi tipe, dan relasi. |
| Class Diagram | Bagian 3 menyajikan diagram ringkas, kardinalitas, dan sumber PlantUML lengkap. |
| Atribut dan access modifier | Lampiran A dan B mencantumkan seluruh atribut dalam lingkup model beserta tipe dan visibilitasnya. |
| Method dan access modifier | Lampiran B memuat public atau private, parameter, dan tipe hasil untuk seluruh operasi kelas. |
| Create code tanpa detail method | 11 kerangka kelas dibuat dan lulus pemeriksaan tipe strict; isi method belum diimplementasikan. |
| Usahakan login berjalan | Implementasi login tersedia pada kode. Verifikasi Google asli dan konfigurasi cloud masih diperlukan. |
| Laporan PDF | Disusun dengan sistematika dokumen acuan dan disertai salinan DOCX yang dapat diedit. |

Rancangan memperbaiki kekurangan model awal dengan memisahkan data, kebijakan akses, layanan, dan penyimpanan. Hubungan dokumen mengikuti implementasi Firebase Inventora. Kerangka kelas menjadi hasil perancangan yang dapat diperiksa, sedangkan penerapan kelas ke komponen React dan verifikasi login Google asli belum dinyatakan selesai.

## Lampiran A. Deklarasi tipe data

Deklarasi berikut memuat 15 interface data dalam lingkup diagram. Seluruh anggotanya public secara semantik TypeScript. Nama PaymentInfo diberikan untuk map PurchaseOrder.paymentInfo agar relasinya dapat ditampilkan sebagai tipe tersendiri. Tipe berakhiran ? bersifat opsional.

### A.1 Alias yang dipakai model

```
export type Role = 'Super Admin'
  | 'Warehouse Manager'
  | 'Finance Manager' | 'HR Manager';
export type POStatus = 'Pending Approval'
  | 'Disetujui' | 'Barang Diterima'
  | 'Selisih Penerimaan' | 'Selesai' | 'Ditolak';
export type ThreeWayMatchStatus = 'MATCHED'
  | 'DISCREPANCY' | 'PENDING_RECEIPT'
  | 'PENDING_INVOICE';
export type SubscriptionTier =
  'starter' | 'business' | 'enterprise';
export type BillingCycle = 'monthly' | 'annual';
```

### A.2 UserSession

Sumber: docs/tubes-class-diagram/kerangka.ts.

```
export interface UserSession {
  uid: string;
  email: string | null;
  displayName: string | null;
  role: Role | null;
  isDemo: boolean;
}
```

### A.3 PurchaseOrder

Sumber: src/types.ts.

```
export interface PurchaseOrder {
  id: string;
  vendor: string;
  desc: string;
```

```
  date: string;
  deliveryDate?: string;
  total: number;
  status: POStatus | string;
  color: 'orange' | 'green' | 'blue' | 'red' | string;
  department?: 'IT Infrastructure' | 'Operations & Logistics' |
    'Engineering & R&D' | 'HR & GA';
  items?: POLineItem[];
  lineItems?: any[];
  subtotal?: number;
  discountAmount?: number;
  taxAmount?: number;
  withholdingTaxAmount?: number;
  grn?: GoodsReceiptNote;
  threeWayMatchStatus?: ThreeWayMatchStatus;
  paymentInfo?: PaymentInfo;
}
```

**A.4 POLineItem**

Sumber: src/types.ts.

```
export interface POLineItem {
  id: string;
  sku?: string;
  name: string;
  qty: number;
  unitPrice: number;
  subtotal: number;
}
```

### A.5 GoodsReceiptNote

Sumber: src/types.ts.

```
export interface GoodsReceiptNote {
  id: string;
  grnNumber: string;
  poId: string;
  receivedDate: string;
  receivedBy: string;
  suratJalanNumber: string;
  items: GoodsReceiptItem[];
  status: 'MATCHED' | 'DISCREPANCY' | 'PENDING';
  inspectorNotes?: string;
}
```

### A.6 GoodsReceiptItem

Sumber: src/types.ts.

```
export interface GoodsReceiptItem {
  sku?: string;
  name: string;
  orderedQty: number;
  receivedQty: number;
  condition: 'EXCELLENT' | 'GOOD' | 'DAMAGED' | 'SHORTAGE';
  notes?: string;
}
```

### A.7 PaymentInfo

Sumber: src/types.ts: PurchaseOrder.paymentInfo.

```
export interface PaymentInfo {
  paidAt: string;
  method: string;
  txId: string;
  reference: string;
  amount: number;
  accountOrCard: string;
  receiptNo: string;
  payerName: string;
}
```

### A.8 PaymentTransaction

Sumber: src/types.ts.

```
export interface PaymentTransaction {
  id: string;
  poId: string;
  vendor: string;
  amount: number;
  taxAmount: number;
  withholdingTaxAmount?: number;
  totalPaid: number;
  method: 'bank_transfer' | 'virtual_account' | 'credit_card' |
    'qris';
  methodLabel: string;
  accountReference: string;
  status: 'SUCCESS' | 'PENDING' | 'FAILED';
  timestamp: string;
  receiptNumber: string;
  notes?: string;
}
```

### A.9 DepartmentBudget

Sumber: src/types.ts.

```
export interface DepartmentBudget {
  id: string;
  name: string;
  code: string;
  quarter: string;
  allocated: number;
  spent: number;
  committed: number;
  currency: string;
}
```

### A.10 Employee

Sumber: src/components/HRView.tsx.

```
export interface Employee {
  id: string;
  name: string;
  email: string;
  dept: string;
  role: string;
  status: string;
  color: 'green' | 'gray' | 'red';
  joined: string;
  taxId?: string;
  salary: EmployeeSalary;
}
```

### A.11 EmployeeSalary

Sumber: src/components/HRView.tsx.

```
export interface EmployeeSalary {
  baseSalary: number;
  positionAllowance: number;
  transportAllowance: number;
```

```
  bonus: number;
  bpjsKesehatan: number;
  bpjsKetenagakerjaan: number;
  pph21: number;
  bankName: string;
  accountNumber: string;
  disbursementStatus: 'Disbursed' | 'Scheduled' |
    'Pending Approval';
  lastDisbursed?: string;
  disbursementRef?: string;
}
```

**A.12 PayrollDisbursementRecord**

Sumber: src/components/HRView.tsx.

```
export interface PayrollDisbursementRecord {
  id: string;
  referenceNumber: string;
  empId: string;
  empName: string;
  dept: string;
  role: string;
  taxId?: string;
  period: string;
  periodKey?: string;
  disbursedAt: string;
  disbursedDateFormatted: string;
  baseSalary: number;
  allowances: number;
  positionAllowance?: number;
  transportAllowance?: number;
  bpjsKesehatan?: number;
  bpjsKetenagakerjaan?: number;
  bonus: number;
  gross: number;
  deductions: number;
```

```
  pph21: number;
  bpjs: number;
  netPay: number;
  bankName: string;
  accountNumber: string;
  actor: string;
  status: 'SUCCESS' | 'SCHEDULED';
}
```

**A.13 InventoryItem**

Sumber: docs/tubes-class-diagram/kerangka.ts.

```
export interface InventoryItem {
  id: string;
  name: string;
  desc: string;
  stock: number;
  status: string;
  color: 'orange' | 'green' | 'blue' | 'red';
}
```

**A.14 InvoiceRecord**

Sumber: docs/tubes-class-diagram/kerangka.ts.

```
export interface InvoiceRecord {
  id: string;
  client: string;
  desc: string;
  date: string;
  total: number;
  status: string;
  color: 'orange' | 'green' | 'blue' | 'red';
  isOverdue: boolean;
}
```

### A.15 SubscriptionInvoice

Sumber: docs/tubes-class-diagram/kerangka.ts.

```
export interface SubscriptionInvoice {
  id: string;
  date: string;
  total: number;
  period: string;
  status: string;
  client: string;
  planName: string;
  paymentMethod: string;
}
```

### A.16 SubscriptionRecord

Sumber: docs/tubes-class-diagram/kerangka.ts.

```
export interface SubscriptionRecord {
  planId: SubscriptionTier;
  cycle: BillingCycle;
  anchorDate: string;
  autoRenew: boolean;
  invoices: SubscriptionInvoice[];
}
```

## Lampiran B. Kerangka kelas

Kode lengkap yang dapat diperiksa berada pada kerangka.ts. Cuplikan per kelas berikut mempertahankan access modifier, parameter, tipe hasil, dan penanda method yang sama. Tipe Auth, GoogleAuthProvider, User, Firestore, Unsubscribe, dan Storage berasal dari SDK Firebase atau API browser. Tipe data domain dijelaskan pada Lampiran A.

### B.1 Alias parameter dan listener

```
export type MutationSource =
  'approval' | 'receipt' | 'payment';
export type PoAction =
  'create' | 'approve' | 'receive' | 'pay';
export type SessionListener =
  (session: UserSession | null) => void;
export type PurchaseOrderListener =
  (orders: PurchaseOrder[]) => void;
```

### B.2 AuthService

```
export class AuthService {
  private provider!: GoogleAuthProvider;
  private cachedAccessToken: string | null = null;
  public constructor(private readonly auth: Auth) {}
  public signIn(): Promise<UserSession | null> {
    throw new Error('Belum diimplementasikan');
  }
  public observe(listener: SessionListener): Unsubscribe {
    throw new Error('Belum diimplementasikan');
  }
  public getGoogleAccessToken(
    forceRefresh?: boolean,
  ): Promise<string | null> {
    throw new Error('Belum diimplementasikan');
  }
  public signOut(): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
```

```
  private mapUser(user: User, role: Role | null): UserSession {
    throw new Error('Belum diimplementasikan');
  }
}
```

**B.3 AuthorizationPolicy**

```
export class AuthorizationPolicy {
  public isRole(value: unknown): value is Role {
    throw new Error('Belum diimplementasikan');
  }
  public canAccessMenu(role: Role | null, menu: string):
    boolean {
    throw new Error('Belum diimplementasikan');
  }
  public canPerformPoAction(
    role: Role | null,
    action: PoAction,
  ): boolean {
    throw new Error('Belum diimplementasikan');
  }
}
```

**B.4 LoginController**

```
export class LoginController {
  private session: UserSession | null = null;
  public constructor(
    private readonly authService: AuthService,
    private readonly policy: AuthorizationPolicy,
  ) {}
  public loginWithGoogle(): Promise<UserSession | null> {
    throw new Error('Belum diimplementasikan');
  }
  public logout(): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
```

```
  public canEnterDashboard(): boolean {
    throw new Error('Belum diimplementasikan');
  }
}
```

**B.5 PurchaseOrderModel**

```
export class PurchaseOrderModel {
  public constructor(private readonly data: PurchaseOrder) {}
  public getData(): Readonly<PurchaseOrder> {
    throw new Error('Belum diimplementasikan');
  }
  public calculateTotal(): number {
    throw new Error('Belum diimplementasikan');
  }
  public approve(role: Role): void {
    throw new Error('Belum diimplementasikan');
  }
  public reject(role: Role): void {
    throw new Error('Belum diimplementasikan');
  }
  public receive(note: GoodsReceiptNote, role: Role): void {
    throw new Error('Belum diimplementasikan');
  }
  public recordPayment(info: PaymentInfo, role: Role): void {
    throw new Error('Belum diimplementasikan');
  }
}
```

### B.6 GoodsReceiptModel

```
export class GoodsReceiptModel {
  public constructor(private readonly data: GoodsReceiptNote) {}
  public getData(): Readonly<GoodsReceiptNote> {
    throw new Error('Belum diimplementasikan');
  }
  public hasDiscrepancy(): boolean {
    throw new Error('Belum diimplementasikan');
  }
}
```

### B.7 PurchaseOrderRepository

```
export class PurchaseOrderRepository {
  public constructor(private readonly db: Firestore) {}
  public create(po: PurchaseOrder): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
  public findById(id: string): Promise<PurchaseOrder | null> {
    throw new Error('Belum diimplementasikan');
  }
  public listen(listener: PurchaseOrderListener): Unsubscribe {
    throw new Error('Belum diimplementasikan');
  }
  public transition(
    id: string, patch: Partial<PurchaseOrder>,
    source: MutationSource, mutationId?: string,
  ): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
}
```

**B.8 ProcurementService**

```
export class ProcurementService {
  public constructor(
    private readonly repository: PurchaseOrderRepository,
    private readonly policy: AuthorizationPolicy,
  ) {}
  public create(po: PurchaseOrder, role: Role): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
  public approve(id: string, role: Role): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
  public reject(id: string, role: Role): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
  public receive(
    id: string, note: GoodsReceiptNote, role: Role,
  ): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
  public pay(
    id: string, info: PaymentInfo, role: Role,
  ): Promise<void> {
    throw new Error('Belum diimplementasikan');
  }
}
```

**B.9 InventoryService**

```
export class InventoryService {
  public constructor(private readonly storage: Storage) {}
  public list(): InventoryItem[] {
    throw new Error('Belum diimplementasikan');
  }
  public deploy(sku: string, qty: number): void {
    throw new Error('Belum diimplementasikan');
  }
}
```

**B.10 PayrollService**

```
export class PayrollService {
  public constructor(private readonly storage: Storage) {}
  public listEmployees(): Employee[] {
    throw new Error('Belum diimplementasikan');
  }
  public getRecord(
    empId: string, periodKey: string,
  ): PayrollDisbursementRecord | undefined {
    throw new Error('Belum diimplementasikan');
  }
  public record(
    empId: string, periodKey: string,
  ): PayrollDisbursementRecord {
    throw new Error('Belum diimplementasikan');
  }
}
```

### B.11 FinanceService

```
export class FinanceService {
  public constructor(private readonly storage: Storage) {}
  public listInvoices(): InvoiceRecord[] {
    throw new Error('Belum diimplementasikan');
  }
  public markAsPaid(id: string): InvoiceRecord {
    throw new Error('Belum diimplementasikan');
  }
}
```

### B.12 BillingService

```
export class BillingService {
  public constructor(private readonly storage: Storage) {}
  public getSubscription(): SubscriptionRecord {
    throw new Error('Belum diimplementasikan');
  }
  public changePlan(
    planId: SubscriptionTier, cycle: BillingCycle,
  ): SubscriptionRecord {
    throw new Error('Belum diimplementasikan');
  }
  public setAutoRenew(enabled: boolean): void {
    throw new Error('Belum diimplementasikan');
  }
}
```

**Lampiran C. Berkas dan cara pemeriksaan**

| Berkas | Kegunaan |
|---|---|
| docs/tubes-class-diagram/kerangka.ts | 11 kelas dan tipe lokal yang dapat diperiksa TypeScript. |
| docs/tubes-class-diagram/data-models.ts | Salinan deklarasi SDM dan paket langganan dari komponen. |
| docs/tubes-class-diagram/tsconfig.json | Konfigurasi pemeriksaan strict kerangka. |
| docs/tubes-class-diagram/Inventora_Class_Diagram.puml | Sumber UML lengkap dengan seluruh anggota dan relasi. |
| docs/tubes-class-diagram/model.json | Metadata deklarasi dan hash sumber yang membentuk diagram. |
| docs/tubes-class-diagram/README.md | Petunjuk pemeriksaan dan batas implementasi. |
| database/firestore/hasil-uji.json | Bukti 18 pemeriksaan emulator dari tugas basis data sebelumnya. |

### C.1 Perintah pemeriksaan

Jalankan dari folder proyek dengan Node.js dan dependensi yang terpasang. Perintah tidak mengimplementasikan method dan tidak menjalankan login Google.

```
node --import tsx --test tests/regressions.test.ts
npx tsc --noEmit
npx tsc -p docs/tubes-class-diagram/tsconfig.json
```

### C.2 Pemutakhiran model

```
node docs/tubes-class-diagram/susun-model.mjs
```

Perintah membentuk kembali model.json, data-models.ts, dan sumber PlantUML berdasarkan deklarasi yang dipilih. Jika sumber aplikasi berubah, hasil diagram dan laporan perlu diperiksa kembali. Dokumen ini menggambarkan keadaan kode pada tanggal pemeriksaan, bukan daftar fitur yang seluruhnya sudah dipindahkan ke cloud.